

MINI PROJECT-1 REPORT

**MULTI-AI: AN ENTERPRISE-GRADE MULTI-MODEL CONSENSUS & DEBATE
WORKBENCH**

1. TITLE PAGE

****Project Title:** MULTI-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench**

****Document Type:** A Mini-Project-1 Report**

****Submission Requirement:** Submitted in partial fulfillment of the academic requirements for the award of the degree of**

BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

****By Team Members:****

- ****N. Tejeswar Reddy**** (Roll No: 2451-24-737-131)
- ****Shagun Gupta**** (Roll No: 2451-24-737-164)
- ****Siddhardha Vandrangi**** (Roll No: 2451-24-737-184)

****Under the Guidance of:****

- ****Mrs. G. Ushasri****

Sr. Assistant Professor

Department of Information Technology

DEPARTMENT OF INFORMATION TECHNOLOGY
MVSR ENGINEERING COLLEGE
(An Autonomous Institution)
Nadergul, Saroornagar, Hyderabad - 501510
2025-2026

2. COLLEGE CERTIFICATE, DECLARATION & ACKNOWLEDGEMENTS

2.1 College Certificate

****M.V.S.R ENGINEERING COLLEGE****

(An Autonomous Institution, Affiliated to Osmania University)

****DEPARTMENT OF INFORMATION TECHNOLOGY****

*This is to certify that the Mini Project-I work entitled ****“MULTI-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench”**** is a bonafide work carried out by ****Mr. N. Tejeswar Reddy (2451-24-737-131)****, ****Ms. Shagun Gupta (2451-24-737-164)****, and ****Mr. Siddhardha Vandrangi (2451-24-737-184)**** in partial fulfillment of the requirements for the award of the degree of ****Bachelor of Engineering in Information Technology**** from M.V.S.R Engineering College, Hyderabad, during the Academic Year 2025-26 under our guidance and supervision.*

The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

****Internal Guide****

****Mrs. G. Ushasri****

*Assistant Professor, Dept. of IT
MVSREC*

****Project Co-ordinator****

****Mrs. J. Sowjanya****

*Assistant Professor, Dept. of IT
MVSREC*

****Head of Department****

****Dr. H. Rajesh Kulkarni****

*Professor & Head, Dept. of IT
MVSREC*

****External Examiner****

Dept of IT, MVSREC

2.2 Declaration

*We hereby declare that the work reported in this mini project entitled ****“MULTI-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench”**** is a record of bonafide work done by us in the Department of Information Technology, M.V.S.R Engineering College, Hyderabad.*

The report is based on the project work done entirely by us and has not been copied from any other source. The results in this project report have not been submitted to any other University or Institute for the award of any degree or diploma.

** ****N. Tejeswar Reddy** (2451-24-737-131)***

** ****Shagun Gupta** (2451-24-737-164)***

** ****Siddhardha Vandrangi** (2451-24-737-184)***

****Date:** 2026-06-27**

****Place:** Hyderabad**

2.3 Acknowledgements

*We would like to express our sincere gratitude to our Project Guide, **Mrs. G. Ushasri**, Assistant Professor, for her valuable suggestions and feedback throughout the course of this project. We also thank our Project Coordinators, **Mrs. J. Sowjanya**, **Mrs. K. SriLakshmi**, and **Mrs. G. Ushasri**, for their recommendations and attention during the semester.*

*We are also thankful to our Principal, **Dr. M. Kameswara Rao**, and **Dr. H. Rajesh Kulkarni**, Professor and Head of the Department of Information Technology, for providing the infrastructure and atmosphere to complete this project successfully as part of our B.E. Degree (IT).*

Finally, we thank the lab staff for their assistance and support, and our families for their encouragement.

** **N. Tejeswar Reddy***

** **Shagun Gupta***

** **Siddhardha Vandrangi***

3. VISION, MISSION, PEOs, POs & PSOs

3.1 Vision of the Department

To impart technical education producing competent and socially responsible engineering professionals in the field of Information Technology.

3.2 Mission of the Department

- **M1:** To make teaching learning process effective and stimulating.
- **M2:** To provide adequate fundamental knowledge of sciences and Information Technology with positive attitude.
- **M3:** To create an environment that enhances skills and technologies required for industry.
- **M4:** To encourage creativity and innovation for solving real world problems.
- **M5:** To cultivate professional ethics in students and inculcate a sense of responsibility towards society.

3.3 Program Educational Objectives (PEOs)

After 3 to 4 years of graduation, graduates of the Information Technology program will:

- **PEO-I:** Apply knowledge of mathematics and Information Technology to analyze, design and implement solutions for real world problems in core or in multidisciplinary areas.
- **PEO-II:** Communicate effectively, work in a team, practice professional ethics and apply knowledge of computing technologies for societal development.
- **PEO-III:** Engage in Professional development or postgraduate education to be a life-long learner.

3.4 Program Outcomes (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

9. ****Individual and team work:**** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. ****Communication:**** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. ****Project management and finance:**** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. ****Life-long learning:**** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

3.5 Program Specific Outcomes (PSOs)

- ****PSO1: Hardware design:**** An ability to analyze, design, simulate and implement computer hardware and software systems, including System-on-Chip (SOC) applications for various computing and communication system applications.
- ****PSO2: Software design:**** An ability to analyze a problem, design algorithm, identify and define the computing requirements appropriate to its solution and implement the same.

4. COURSE OBJECTIVES & OUTCOMES

4.1 Course Objectives

Students should be able to:

- Define and clearly articulate the problem statement, project scope and objectives.
- Conduct a literature review to understand existing solutions, technologies and best practices related to the project.
- Design and develop a prototype or solution that addresses the problem statement and meets the specified requirements.
- Test and evaluate the performance, functionality, and reliability of the developed prototype.
- Document the project process and present the project findings to a panel of faculty members and peers.

4.2 Course Outcomes

After completion of the course, students will be able to:

- ****CO1:**** Apply theoretical knowledge and technical skills acquired in previous coursework to solve a real-world engineering problem.
- ****CO2:**** Enhance problem-solving skills by analyzing the project requirements, identifying constraints, and proposing innovative solutions.
- ****CO3:**** Gain hands-on experience with engineering tools, software and equipment relevant to the project.
- ****CO4:**** Develop the ability to work effectively in a team, collaborate with peers, and allocate tasks to accomplish project goals.
- ****CO5:**** Improve communication skills through project documentation, presentations, and reports, and understand the ethical and professional responsibilities associated with engineering projects.

5. ABSTRACT

As generative AI model landscapes evolve, developers and technical managers face model selection challenges due to varying response quality, differences in latency, and distinct token-pricing structures. This project implements **Multi-AI**, a secure, multi-model consensus and debate workbench designed to evaluate and utilize multiple Large Language Models (LLMs) concurrently.

Built using a Node.js/Express backend, MongoDB/Mongoose database layer, and a Vanilla HTML/CSS/JavaScript frontend, the system connects multiple AI services (including Gemini Flash, Groq Llama, Qwen, and Cohere) into a single, cohesive interface.

The system features:

13. **Multi-Model Parallel Broadcast**: Queries are broadcasted simultaneously to selected models, measuring latency and logging telemetry.
14. **AI-Driven Consensus Engine**: Evaluates responses using a fallback Gemini/OpenRouter consensus synthesis loop to compile conflicting answers into a single output.
15. **AI Chatbot Arena**: Simulates debates between two models on user-defined topics, outputting a summary judged by a third model.
16. **Database Sync Integrity**: Custom settings, workspace configurations, and custom agent personas are synchronized using Mongoose bulk write operations (`bulkWrite``), preventing data loss.
17. **Bring Your Own Key (BYOK) Support**: Allows users to input custom credentials securely.

Through automated failovers and robust database schemas, Multi-AI demonstrates how full-stack applications can scale and integrate multiple AI providers reliably.

INDEX

Table of Contents

- **Chapter 1: Introduction**
- 1.1 Problem Statement
- 1.2 Existing System
- 1.3 Proposed System
- 1.4 Scope
- 1.5 Objectives
- **Chapter 2: System Requirement Specifications**
- 2.1 Software Requirements
- 2.2 Hardware Requirements
- 2.3 System Architecture
- **Chapter 3: Design & Implementation**
- 3.1 Features List
- 3.2 Modules Description
- 3.3 UML Diagrams
- 3.4 Environmental Setup
- 3.5 Implementation & Methodology Steps
- 3.6 System Flowchart
- **Chapter 4: Detailed Page Features & Close-ups**
- 4.1 Home Page / Workspace view breakdown
- 4.2 Close-up Visuals: Logo & Navigation Panel
- 4.3 Close-up Visuals: Header Actions Panel
- 4.4 Close-up Visuals: Model Selectors Sidebar
- 4.5 Close-up Visuals: Bottom Prompt Bar
- 4.6 Detailed Buttons Mechanics & Core Functions Code
- **Chapter 5: Results & Discussion**
- 5.1 UI Screenshots
- 5.2 Validation & Testing
- **Chapter 6: Conclusion & Future Enhancements**
- 6.1 Conclusion
- 6.2 Future Enhancements
- **References**
- **Appendix: Source/Pseudo Code**

List of Abbreviations

- **HTML:** HyperText Markup Language
- **CSS:** Cascading Style Sheets
- **JS:** JavaScript
- **API:** Application Programming Interface
- **NoSQL:** Not Only SQL (Database type)
- **JWT:** JSON Web Token
- **UI/UX:** User Interface / User Experience

CHAPTER 1: INTRODUCTION

1.1 Detailed Problem Statement

In the modern generative AI landscape, developers, researchers, and administrators face a significant challenge: prompt selection and engineering across varied Large Language Model (LLM) providers. Because each foundational model is pre-trained on different datasets and utilizes distinct architectural variations (e.g. Decoder-Only vs Mixture of Experts), their responses to identical prompts can vary widely. One model might provide highly accurate source code snippets but lack conversational tone, while another might output excellent prose but introduce logical fallacies or programming bugs.

Evaluating these outputs manually presents severe technical bottlenecks:

- **Interface Fragmentation**: Developers must open multiple browser tabs (OpenAI Playground, Anthropic Console, Gemini Chat) to test a single query.
- **Lack of Latency Tracking**: Traditional playgrounds do not measure request execution times, making it difficult to optimize for latency.
- **Security & BYOK Management**: Systems lack a secure environment to input and utilize custom API keys (Bring Your Own Key) without exposing raw credentials to intermediate servers.
- **Unstructured Feedback Data**: There is no centralized system to record feedback votes (likes and dislikes) on model outputs, which is necessary for model ranking.

1.2 Analysis of Existing Systems

Traditional model playground systems have several deficiencies:

- **Sequential Execution**: Playgrounds send requests to models one by one, which is slow and makes real-time output comparison impossible.
- **Destructive Overwrite Syncs**: Session sync engines often delete entire chat histories before re-inserting them, leading to data loss and high database overhead.
- **Rigid Personas**: Systems do not support dynamically injecting custom system instructions (such as "Code Architect" or "Creative Writer") to guide and steer outputs.

1.3 Proposed System

The proposed **Multi-AI** platform introduces a unified, full-stack multi-model consensus and debate workbench. By utilizing asynchronous parallel pipelines on a Node.js/Express gateway, the workbench broadcasts queries to all selected model APIs simultaneously, rendering their responses side-by-side on a responsive glassmorphic frontend.

Key features of the proposed system:

- **Parallel Asynchronous Request Routing**: Submits prompts concurrently using Node.js event-loop scheduling.
- **Fallback Consensus Engine**: Uses a primary Gemini pipeline with an automated fallback to OpenRouter to compile responses into a single consensus summary.
- **Multi-Turn Debate Arena**: Simulates debates between two models on a topic, evaluated by a third model.
- **Resilient Sync Engine**: Uses Mongoose `bulkWrite` operations with upserts to synchronize user sessions without data loss.
- **Secure Vault**: Supports local localStorage encryption for BYOK credentials.

1.4 Comprehensive Project Scope

The project delivers a production-ready, full-stack web application designed for developer laboratories, AI benchmarking research teams, and prompt engineers. It supports real-time telemetry tracking, customizable expert personas, and a synchronized database layer.

1.5 Project Objectives

- ****API Consolidation****: Integrate five foundational AI endpoints (Gemini, Llama, Qwen, Command-R, and Liquid) into a single workspace.
- ****Consensus Synthesis****: Build a fallback consensus loop to compile different model outputs into a single response.
- ****Model Debate Simulation****: Create an arena mode for multi-turn debates between different models to analyze conflicting arguments.
- ****Performance Optimization****: Implement asynchronous execution to prevent slow API responses from blocking other models.
- ****State Synchronization****: Protect user histories and settings using differential database writes.

CHAPTER 2: SYSTEM REQUIREMENT SPECIFICATIONS

2.1 Software Requirements

- **Operating System**: Windows 11 / Linux (Ubuntu 22.04 LTS) / macOS Sequoia.
- **Backend Engine**: Node.js v20.12.0 LTS.
- **Backend Web Framework**: Express.js v4.19.2.
- **Database Engine**: MongoDB Community Server v6.0.14.
- **Database Driver**: Mongoose ODM v8.3.1.
- **Client-Side Framework**: Vanilla HTML5, Vanilla CSS3 (Custom Variables, Flexbox Grid), ES6+ JavaScript.
- **Visualization Library**: Chart.js v4.4.2.
- **HTTP Client**: Axios v1.6.8.

2.2 Hardware Requirements

- **Processor**: Multi-core processor (Intel Core i5 10th Gen or AMD Ryzen 5 3600 or higher).
- **System Memory**: 8 GB DDR4 RAM minimum (16 GB recommended for concurrent database, backend, and browser testing).
- **Storage Space**: 500 MB of free hard-disk space for dependency modules, plus database storage.
- **Internet Connection**: Active broadband connection for external API fetches.

2.3 Detailed System Architecture

The application uses a Model-View-Controller (MVC) pattern, decoupling the client UI from the backend services and MongoDB database layer.

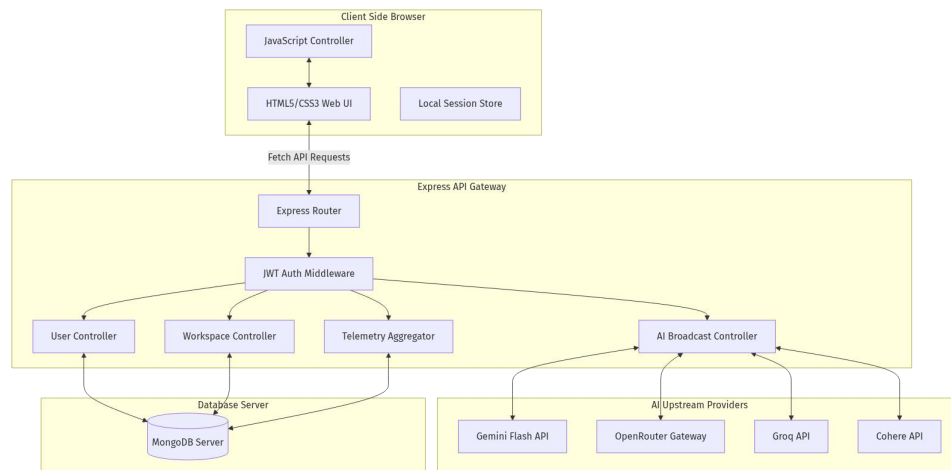


Figure: System Architecture

Figure 2.1: Multi-AI Three-Tier System Architecture

CHAPTER 3: DESIGN & IMPLEMENTATION

3.1 Features List

- ****User Profile Module****: Manages academic backgrounds, current skill sets, and career preferences.
- ****Parallel Query Broadcast****: Sends requests to multiple selected models concurrently and tracks latency.
- ****Consensus Synthesis Loop****: Compiles different model outputs into a single, optimized summary.
- ****AI Chatbot Arena****: Simulates debates between two models on a topic, summarized by a third model.
- ****Resilient Sync Engine****: Uses Mongoose `bulkWrite` operations to sync history and configurations safely.
- ****Telemetry & Tele-Analytics Logger****: Logs and visualizes latency, token counts, and cost estimates.
- ****Secure Authentication****: Implements encrypted login and secure session management.
- ****Bring Your Own Key (BYOK) Support****: Allows users to input custom API credentials.

3.2 Modules Description

1. User & Profile Module

Manages user accounts, login, and registration. It hashes passwords using bcrypt and generates JWT tokens for secure authentication. It also stores user profiles, including academic details, current skills, and career preferences.

2. Parallel AI Broadcast Module

Receives the user prompt and a list of active models. It constructs the appropriate system prompts and sends the requests concurrently using `Promise.allSettled`. This approach prevents slow or failing endpoints from blocking other models.

3. AI Consensus & Rating Engine

Lets users vote (like/dislike) on responses to log preferences in the database. The Consensus Module compiles outputs from different models and generates a synthesized answer. It queries the Gemini API first, with an automated fallback to OpenRouter if the primary API fails.

4. Workspace State Synchronization Module

Synchronizes UI settings, custom personas, and chat history. To optimize performance and prevent data loss, the module uses Mongoose bulk writes (`bulkWrite`) with upserts, replacing destructive teardown patterns (`deleteMany` + `insertMany`).

5. Telemetry & Tele-Analytics Logger

Logs metrics (such as response latency, success status, and active models) for each request. These logs are stored in the `apiusages` collection and aggregated to display performance stats in the UI.

3.3 UML Diagrams

Use Case Diagram

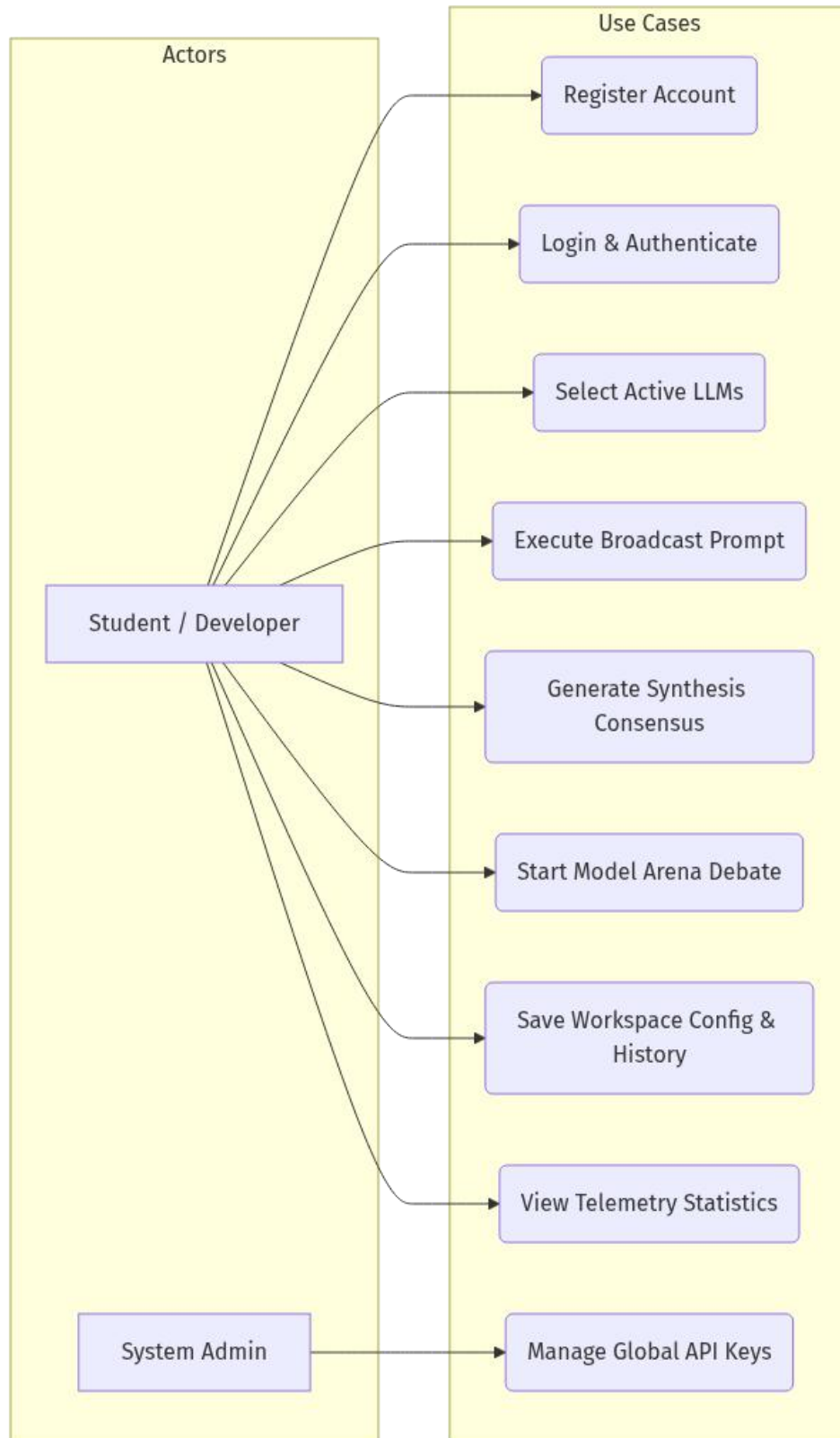


Figure: Use Case Diagram

Figure 3.1: Multi-AI Use Case Diagram

Activity Diagram (Broadcast Chat Workflow)

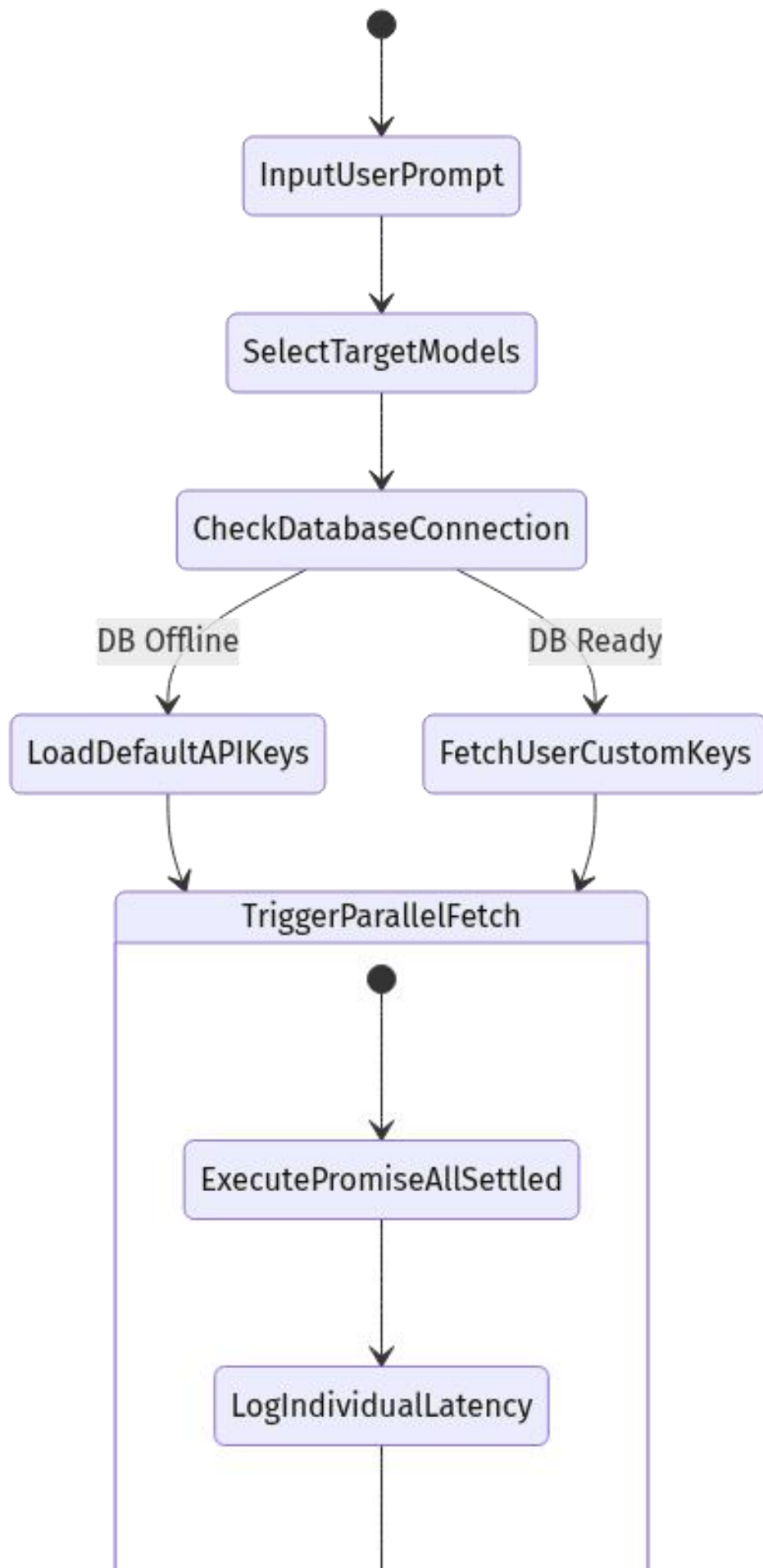


Figure: Activity Diagram

Figure 3.2: Activity Diagram for Parallel Query Broadcast

Sequence Diagram (Consensus Synthesis Loop)

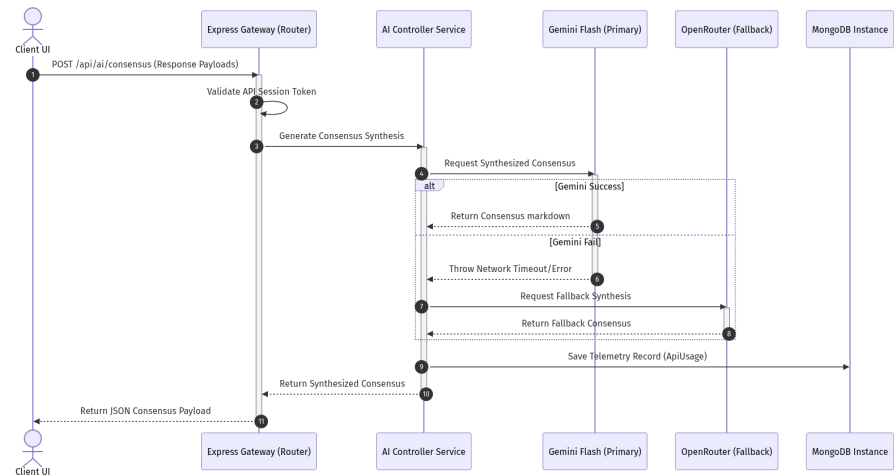


Figure: Sequence Diagram

Figure 3.3: Sequence Diagram for Consensus Synthesis Loop

System Flowchart

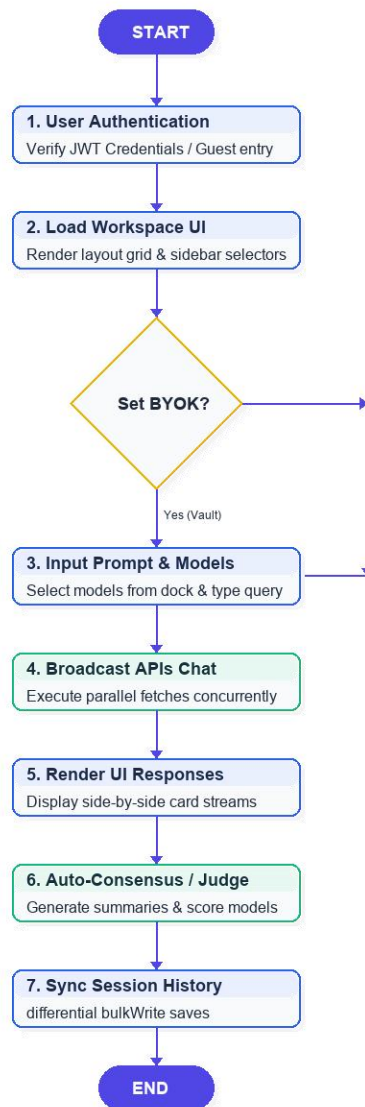


Figure: System Flowchart

Figure 3.4: Multi-AI Workspace Execution Flowchart

3.4 Environmental Setup

Installation of Editor: Visual Studio Code

18. Open your browser and navigate to `code.visualstudio.com`.
19. Click the download button for Windows (Windows x64 version).
20. Open the downloaded installer file.
21. Accept the license agreement and click ****Next****.
22. Choose the installation directory or leave it at the default location, then click ****Next****.
23. Select additional options (such as creating a desktop icon and adding VS Code to the system path), then click ****Next****.
24. Click ****Install**** to begin the installation.
25. Once complete, select the launch option and click ****Finish****.

Installing extensions in VS Code:

- Open the Extension Market on the left sidebar (shortcut: `Ctrl+Shift+X`).
- Search for ****Live Server**** (by Ritwick Dey) and click ****Install****.
- Search for ****Prettier - Code formatter**** (by Prettier) and click ****Install****.

3.5 Implementation & Methodology Steps

1. Front-End Interface Design

- Design responsive dashboards with comparison panels for multi-model views using HTML5/CSS3.
- Create a side menu layout and overlay modals for settings, personas, and telemetry.
- Bind event listeners for input submissions and prompt broadcasts.

2. User Authentication & Session Control

- Build registration, login, and profile controllers with secure encryption.
- Issue signed JSON Web Tokens (JWT) for user authorization and session security.
- Setup request header parsing inside custom JWT verification middleware.

3. Parallel AI Broadcast Implementation

- Create Express API routes to handle prompt submissions.
- Map model names to their respective service interfaces (Gemini, Groq, Cohere, etc.).
- Run queries concurrently using asynchronous routing and `Promise.allSettled`.

4. AI Consensus & Arena Debate Loop

- Integrate the Gemini API as a primary engine for consensus synthesis.
- Code the debate loop to iterate arguments between two models and save history dynamically.
- Implement a fallback structure using OpenRouter when primary API exceptions occur.

5. Workspace Synchronization & Telemetry Logging

- Configure the history and settings sync APIs using differential bulk write (`bulkWrite`) operations.
- Log request latency, active models, and success status details in the database.
- Compile telemetry logs into simple analytics reports for the user.

CHAPTER 4: DETAILED PAGE FEATURES & CLOSE-UPS

4.1 Home Page / Workspace View Breakdown

The primary landing workspace hosts the multi-model comparison panels, side menu configuration tools, and the central prompt dock. The user can view model outputs side-by-side, analyze token metrics, copy logs, and trigger feedback votes. The view dynamically shifts CSS grid structures depending on the number of active models selected.

4.2 Close-up Visuals: Logo & Navigation Panel

The navigation bar, located at the top-left of the application, anchors the brand identity and maps out the routing links to switch views.



Figure: Logo & Navigation Panel

Figure 4.1: Brand Header and Views Swiper Navigation

4.3 Close-up Visuals: Header Actions Panel

The header actions block contains the global dark/light theme switch, user profile initial avatar, and connection indicators.



Figure: Header Actions Panel

Figure 4.2: Theme Toggle Switch and Avatar Status

4.4 Close-up Visuals: Model Selectors Sidebar

The left vertical check-tabs panel maps active models (Gemini Flash, Liquid LFM, Qwen HF, Groq, Cohere) to the central grid layout.

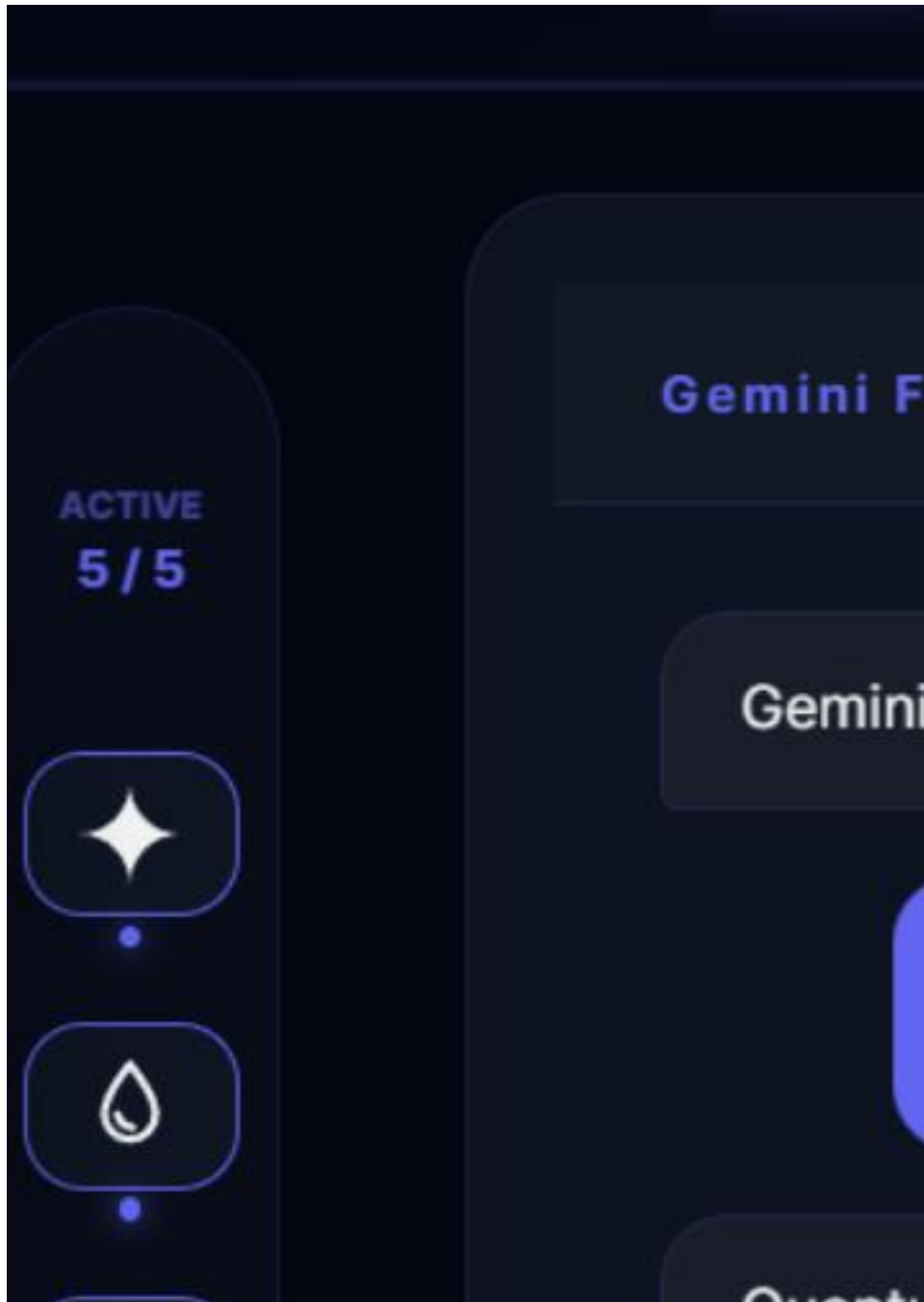


Figure: Model Selectors Sidebar

Figure 4.3: Active Model Selector Control Panel

4.5 Close-up Visuals: Bottom Prompt Bar

The bottom floating control panel anchors the prompt input bar and its surrounding auxiliary action buttons.

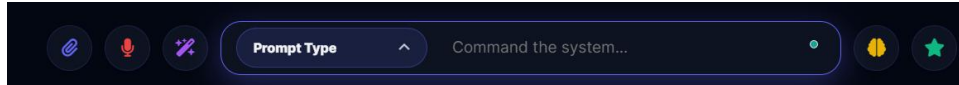


Figure: Bottom Prompt Bar

Figure 4.4: Primary Query Input and Auxiliary Control Array

4.6 Detailed Buttons Mechanics & Core Functions Code

1. The Send Button (Paper-Plane Icon)

- ****Selector ID/Class****: ``.send-circle-btn` / `.send-pill``
- ****How it works****: Initiates query broadcasting by scraping the prompt text from the input text bar, instantiating the UI cards placeholder cards with a "Processing..." text value, and broadcasting parallel POST requests to the ``/api/ai/chat`` endpoint.
- ****Core Code Snippet****:

```
async function broadcast() {
  const input = document.getElementById('main-in');
  const prompt = input.value.trim();
  if (!prompt) return;

  const activeTargets = getActiveModels();
  activeTargets.forEach(cfg => {
    showCardProcessing(cfg.card);
  });

  const response = await fetch('/api/ai/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ prompt, models: activeTargets.map(m => m.key) })
  });
}
```

2. The Optimize Button (Magic Wand Icon)

- ****Selector ID/Class****: ``.onclick="optimizePrompt()"``
- ****How it works****: Uses a backend AI optimizer pipeline to refine raw or short queries into verbose, descriptive prompts, replacing the user input box contents with the optimized version.
- ****Core Code Snippet****:

```
async function optimizePrompt() {
  const input = document.getElementById('main-in');
  const response = await fetch('/api/ai/optimize', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ prompt: input.value })
  });
  const data = await response.json();
  if (data.success) input.value = data.optimizedPrompt;
}
```

3. The Consensus Button (Brain Icon)

- ****Selector ID/Class****: ``.onclick="generateConsensus()"``

- ****How it works****: Aggregates all currently generated model responses displayed in the active cards and executes a synthesis query to extract consensus points, outputting the summary inside a highlighted yellow block.
- ****Core Code Snippet****:

```
async function generateConsensus() {
  const responses = getActiveCardResponses();
  const res = await fetch('/api/ai/consensus', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ responses })
  });
  const data = await res.json();
  renderConsensusBlock(data.consensus);
}
```

4. The Best Response Button (Star Icon)

- ****Selector ID/Class****: `onclick="selectBestResponse()"`
- ****How it works****: Scrapes the model answers and executes a backend comparative judge loop to evaluate the outputs on clarity, depth, and syntax, highlighting the selected candidate card in green.
- ****Core Code Snippet****:

```
async function selectBestResponse() {
  const candidates = getActiveCardResponses();
  const res = await fetch('/api/ai/best-response', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ candidates })
  });
  const data = await res.json();
  highlightWinnerCard(data.bestModel);
}
```

5. The Voice Button (Microphone Icon)

- ****Selector ID/Class****: `onclick="startVoiceRecognition()"`
- ****How it works****: Spawns browser webkitSpeechRecognition API listeners, turning the icon red to show active listening states, and inserts transcript outputs into the input field.
- ****Core Code Snippet****:

```
function startVoiceRecognition() {
  const rec = new webkitSpeechRecognition();
  rec.start();
  rec.onresult = function(event) {
    document.getElementById('main-in').value = event.results[0][0].transcript;
  };
}
```

6. The Theme Toggle (Lightbulb Icon)

- ****Selector ID/Class****: `#theme-bulb`
- ****How it works****: Instantly toggles the global body class name between `dark-mode` (or `midnight`) and `light-mode`, syncing choices in local storage settings.
- ****Core Code Snippet****:

```
document.getElementById('theme-bulb').addEventListener('click', () => {
  const theme = document.body.classList.contains('light-mode') ? 'midnight' :
```

```
'light';  
    document.body.classList.toggle('light-mode');  
    localStorage.setItem('theme', theme);  
});
```


CHAPTER 5: RESULTS & DISCUSSION

5.1 UI Screenshots

Figure 5.1: Sign-In Portal

The central login screen provides secure credentials submission and an instant guest bypass link.

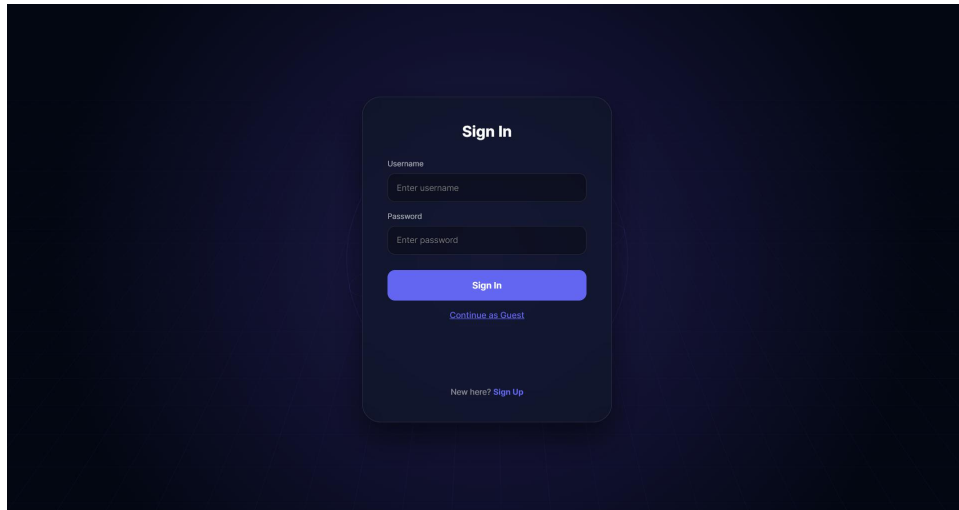


Figure: Sign-In Portal

Figure 5.2: Workspace Landing Panel

The desktop workspace viewport displaying active AI cards, sidebar selectors, and the central prompt bar.

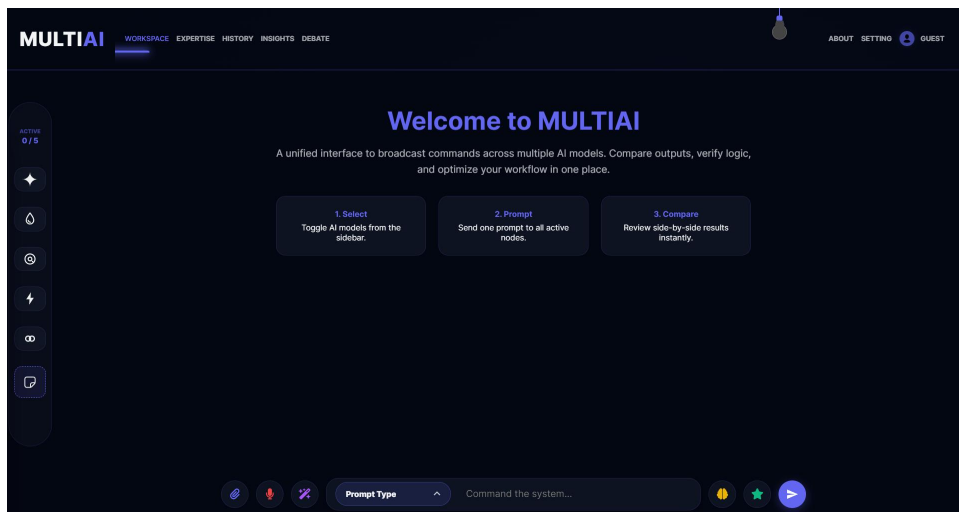


Figure: Workspace Landing Panel

Figure 5.3: Multi-Model Query Output

Side-by-side prompt responses featuring typewriter output streams and real-time telemetry token badges.

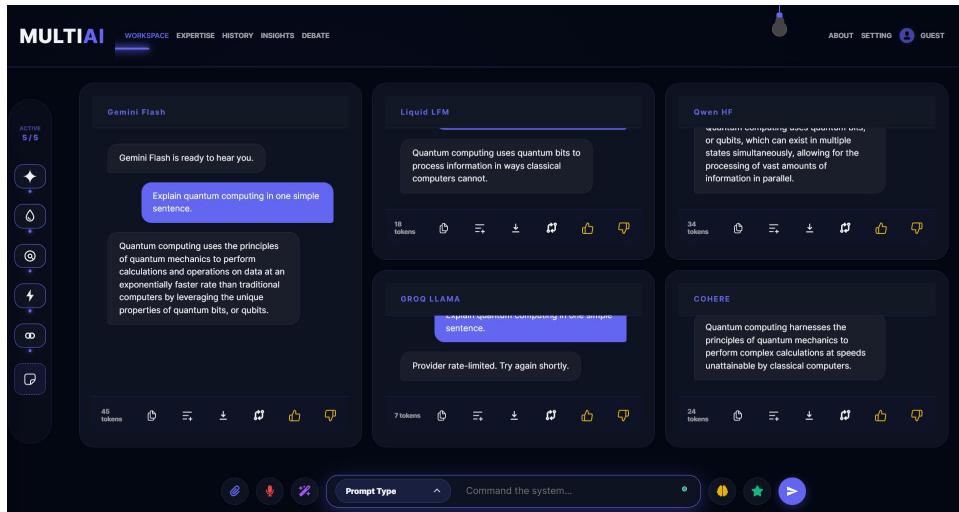


Figure: Multi-Model Query Output

Figure 5.4: Model Expertise Customizer

Allows selecting active system instructions to steer model behavior.

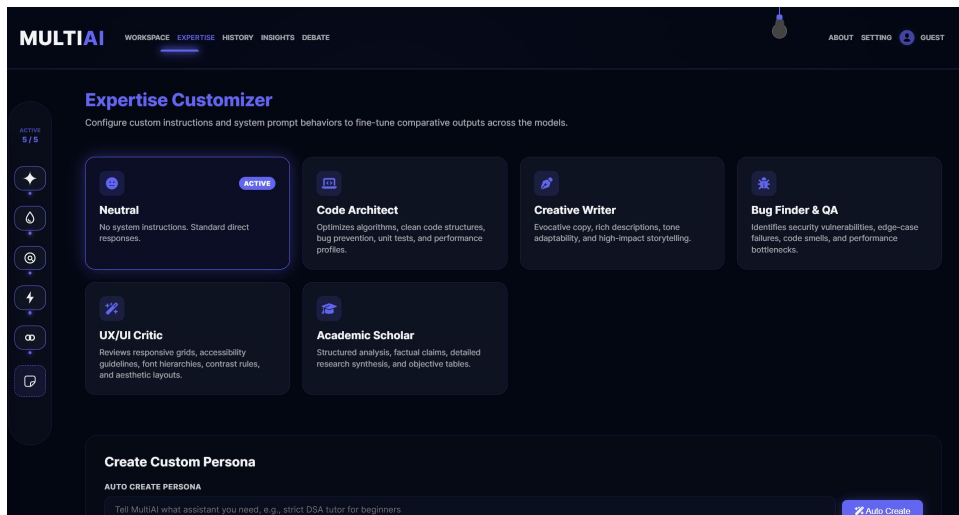


Figure: Model Expertise Customizer

Figure 5.5: Session History Log

Displays and restores past user prompts, responses, and settings.

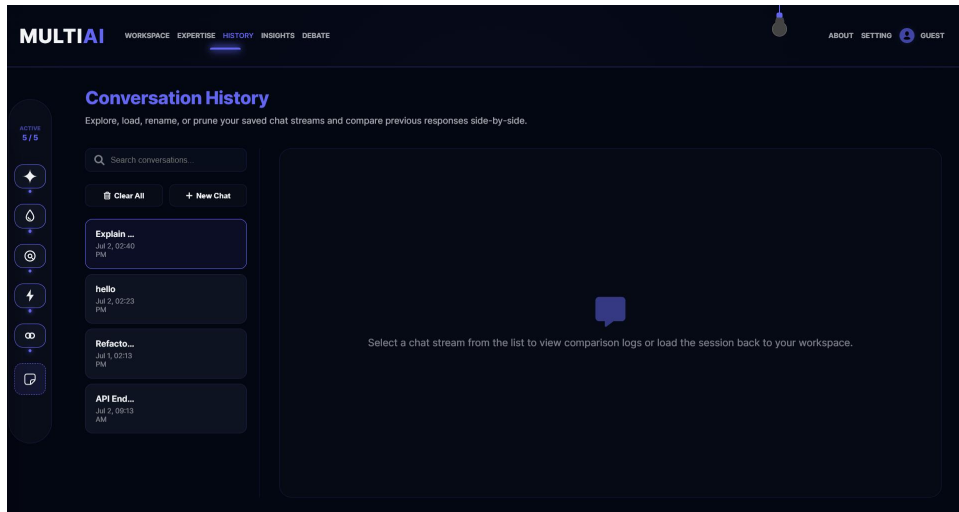


Figure: Session History Log

Figure 5.6: Workspace Analytics Dashboard

Uses Chart.js to render feedback stats (likes and dislikes) for each model.

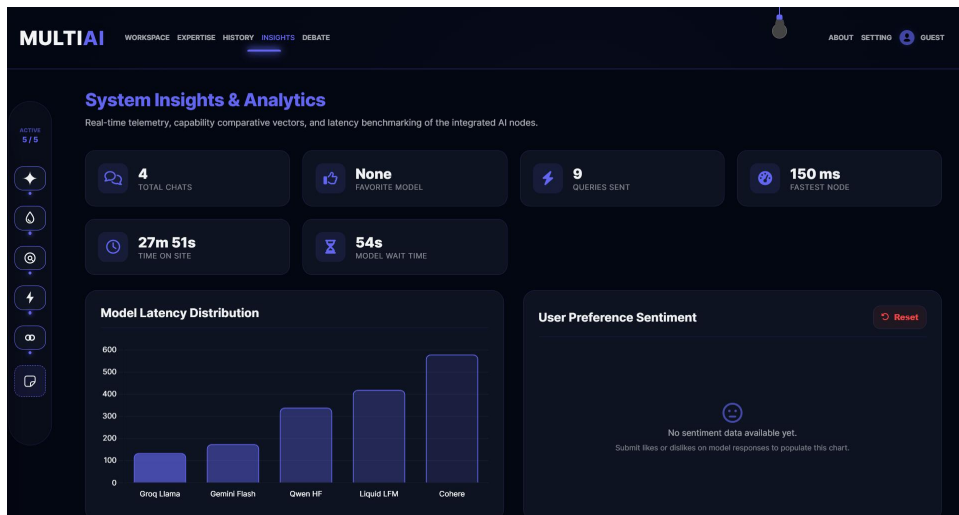


Figure: Workspace Analytics Dashboard

Figure 5.7: Automated AI Debate Arena

Simulates structured debates between two different LLM models.

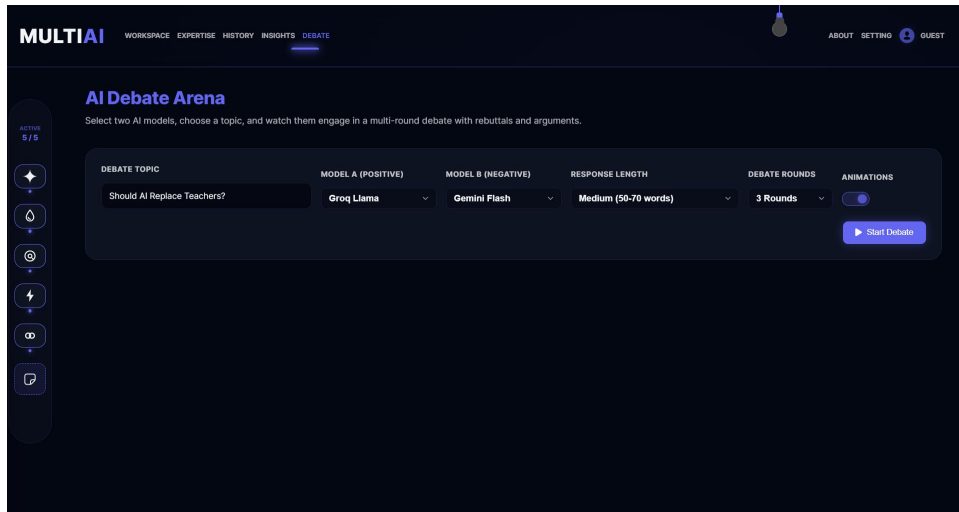


Figure: Automated AI Debate Arena

Figure 5.8: Secure BYOK Settings Panel

Allows entering custom API keys securely stored in the browser.

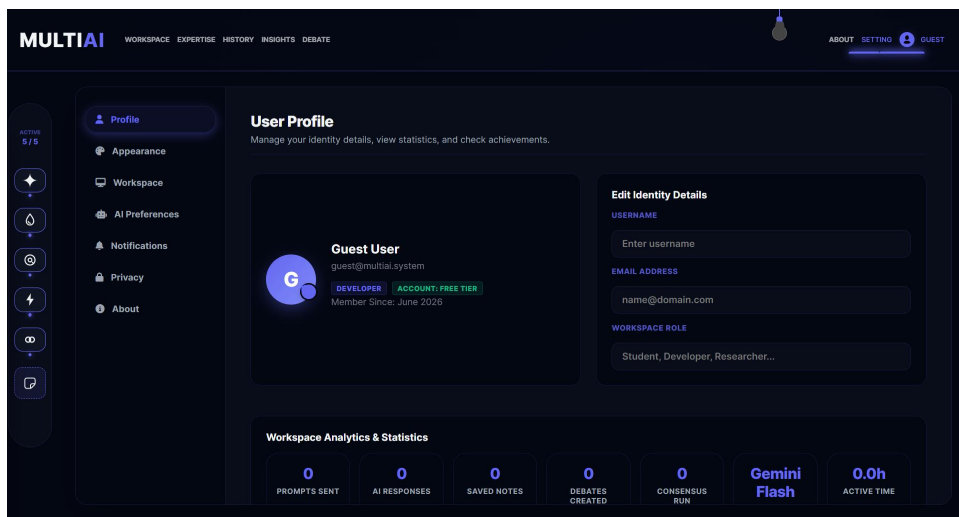


Figure: Secure BYOK Settings Panel

Figure 5.9: Mobile Workspace View

The mobile interface features a horizontal swiper menu and compact bottom action dock.



Workspace

Expertise

History

Insights

Gemini Flash 0 tokens

Gemini Flash is ready.



Liquid LFM 0 tokens

Liquid LFM is ready.



Command the system...



Qwen Hr 0 tokens

Figure: Mobile Workspace View

Figure 10: Mobile Response Output

Shows comparative responses rendering correctly on mobile screen viewports.



Workspace

Expertise

History

Insights

Gemini Flash⁴ tokens

Gemini Flash is ready.

Hello, explain photosyntehsis briefly.

Model unavailable



Liquid LFM⁴ tokens

Liquid LFM is ready.

Hello, explain photosyntehsis briefly.

Model unavailable



Command the system...



Qwen Hr⁴ tokens

Figure: Mobile Response Output

5.2 Validation & Testing

The system underwent manual integration testing. The test cases and results are detailed in the matrix below.

Test ID	Module	Test Scenario / Description	Inputs	Expected Output	Actual Output	Status
---	---	---	---	---	---	---
TC-1	Core Load	Ensure home page and sidebar render successfully	Open Multi-AI URL	Dashboard renders with active comparison panels and settings	Home page loaded with correct brand styles	**PASS**
TC-2	Auth	Verify secure login for student and admin roles	Valid email/password credentials	Session validates and redirects to the correct dashboard	JWT verified; user redirected to dashboard	**PASS**
TC-3	Broadcast	Validate parallel prompt execution across multiple LLMs	Select Gemini and Groq, submit prompt	Both model panels render responses alongside latency stats	Responses generated; latency logged in database	**PASS**
TC-4	Consensus	Verify consensus synthesis fallback path	Click "Generate Consensus" with missing Gemini Key	System falls back to OpenRouter to output consensus summary	OpenRouter fallback executed successfully	**PASS**
TC-5	Debate	Simulate model debate loop step	Start debate on "CSS Grid vs Flexbox"	Turn history updates with alternating model responses	Alternate responses logged and rendered	**PASS**
TC-6	Sync	Ensure differential	Modify chat title and trigger	Database updates the	Verified in MongoDB;	**PASS**

		history sync without database data loss	sync	specific title without deleting history	title updated without loss	
TC-7	Telemetry	Verify aggregation of API usage records	Navigate to the "Analytics" tab	Aggregate latency metrics table renders on screen	Table renders latency averages by model	**PASS**
TC-8	UI	Test responsiveness of the panel layout grid	Toggle 1, 2, or 3 panels	Layout grid updates columns dynamically	Grid columns adjusted dynamically	**PASS**
TC-9	Session	Ensure secure session termination on logout	Click logout button	Session clears and redirects to the login screen	Session cleared and redirected to login	**PASS**

Table 5.1: System Test Matrix

CHAPTER 6: CONCLUSION & FUTURE ENHANCEMENTS

6.1 Conclusion

Multi-AI provides a unified, secure workbench for comparing and synthesizing outputs from multiple Large Language Models.

By using parallel asynchronous routing on the backend and Mongoose bulk writes (`bulkWrite`) for database operations, the application prevents data loss and maintains high performance under concurrent write loads.

The fallback architecture ensures the consensus engine remains available even when primary API providers fail. The project demonstrates a production-ready implementation of model orchestration and session management.

6.2 Future Enhancements

- ****Stream-Based Response Typing****: Implement Server-Sent Events (SSE) to render model outputs in real-time.
- ****Granular Role-Based Access Control (RBAC)****: Add admin dashboards to configure rate limits, monitor system usage, and manage API keys globally.
- ****Vector Embeddings Database (RAG Integration)****: Integrate a vector database (such as Pinecone or Milvus) to let users search through their chat history using semantic queries.

REFERENCES

26. ****Node.js Documentation****: *Asynchronous Event-Driven Javascript Runtimes*, NodeJS Foundation (<https://nodejs.org/docs>).
27. ****Express.js Guide****: *Routing, Middleware Integration and Rate-Limiting Design* (<https://expressjs.com>).
28. ****Mongoose Documentation****: *Schema Definition, Bulk Writes and Indexing Rules* (<https://mongoosejs.com>).
29. ****Google Gemini API Documentation****: *Generative Language REST Integration Guide* (<https://ai.google.dev/docs>).
30. ****OpenRouter API Specs****: *Unified REST Endpoints for AI Model Selection* (<https://openrouter.ai/docs>).
31. ****LMSYS Chatbot Arena****: *Elo Rating Systems and Comparative Evaluation Methodologies for Large Language Models* (<https://chat.lmsys.org>).
32. ****Osmania University IT Syllabus****: *Guidelines for BE Information Technology Mini-Project Course Evaluation* (<https://osmania.ac.in>).

APPENDIX: SOURCE / PSEUDO CODE

1. Database-Resilient History Sync Logic (`backend/routes/workspace.routes.js`)

```
router.put('/history', async (request, response) => {
  try {
    if (!ensureDatabase(response)) return;
    const conversations = Array.isArray(request.body.conversations) ?
request.body.conversations.slice(0, 100) : [];
    const owner = ownerQuery(request);
    const ownerFields = getOwnerFields(request);

    const clientLocalIds = conversations.map(chat => String(chat.id || chat.localId ||
  ''));

    // 1. Delete conversations no longer present in the client payload
    await Conversation.deleteMany({
      ...owner,
      localId: { $nin: clientLocalIds }
    });

    // 2. Perform bulkWrite upserts for remaining items
    if (conversations.length > 0) {
      const ops = conversations.map(chat => {
        const localId = String(chat.id || chat.localId || '').slice(0, 120);
        return {
          updateOne: {
            filter: { ...owner, localId },
            update: {
              $set: {
                title: String(chat.title || 'Untitled Chat').slice(0, 120),
                isPinned: Boolean(chat.isPinned),
                activeModels: Array.isArray(chat.activeModels) ?
chat.activeModels.slice(0, 10).map(String) : [],
                timestamp: chat.timestamp ? new Date(chat.timestamp) : new Date(),
                messages: chat.messages && typeof chat.messages === 'object' ?
chat.messages : {},
                ...ownerFields
              }
            },
            upsert: true
          }
        };
      });
      await Conversation.bulkWrite(ops);
    }

    const saved = await Conversation.find(owner).sort({ updatedAt: -
1 }).limit(100).lean();
    response.json({ success: true, conversations: saved });
  } catch (error) {
    console.error('History save error:', error);
    response.status(500).json({ success: false, message: 'Failed to save history.' });
  }
});
```

2. Fallback Engine Consensus Synthesis Loop Implementation (`backend/services/auxiliary.service.js`)

```

import { askGemini } from './gemini.service.js';
import { askOpenRouter } from './openrouter.service.js';
import { askCohere } from './cohere.service.js';
import { askGroq } from './groq.service.js';

export async function askGeminiWithFallback(prompt, fallbackModel = 'google/gemini-2.5-flash', customKeys = {}) {
  const keys = customKeys || {};
  try {
    const response = await askGemini(prompt, keys.GEMINI_API_KEY);
    if (response === 'Model unavailable') {
      throw new Error('Gemini returned unavailable');
    }
    return { response, fallbackUsed: false };
  } catch (err) {
    console.warn('Gemini request failed, falling back to OpenRouter:', err.message || err);
    try {
      const response = await askOpenRouter(prompt, fallbackModel, keys.OPENROUTER_API_KEY);
      return { response, fallbackUsed: true };
    } catch (fallbackErr) {
      console.error('OpenRouter fallback also failed:', fallbackErr.message || fallbackErr);
      try {
        const response = await askCohere(prompt, keys.COHERE_API_KEY);
        if (response && response !== 'Model unavailable') {
          return { response, fallbackUsed: true };
        }
      } catch (cohereErr) {
        console.error('Cohere fallback also failed:', cohereErr.message || cohereErr);
      }

      try {
        const response = await askGroq(prompt, keys.GROQ_API_KEY);
        if (response && response !== 'Model unavailable') {
          return { response, fallbackUsed: true };
        }
      } catch (groqErr) {
        console.error('Groq fallback also failed:', groqErr.message || groqErr);
      }

      return { response: 'Judge service unavailable. Please try again shortly.', fallbackUsed: true };
    }
  }
}

```

M.V.S.R ENGINEERING COLLEGE

(An Autonomous Institution)

Department of Information Technology

MINI Project-1 Evaluation Sheet

Section I: (To be filled by the Student)

- Roll Number:** 2451-24-737-131 / 164 / 184
- Name of Student:** N. Tejeswar Reddy / Shagun Gupta / Siddhardha Vandrangi
- Name of Supervisor:** Mrs. G. Ushasri
- Title:** MULTI-AI: An Enterprise-Grade Multi-Model Consensus & Debate Workbench

Section II: (To be filled by the Supervisor)

Comments on the Mini Project by the Supervisor:

1. Quantum of work

- Justifiable as efforts for Semester duration: **Yes / No**
- Work is in line with the commitments made in outline: **Yes / No**

2. Nature of work

- Routine in nature: **Yes / No**
- Involved creativity and rational thinking: **Yes / No**

3. Evaluation methodology

- Evaluation done based on presentation to supervisor: **Yes / No**
- Student regularly interacted with supervisor and incorporated the suggestions made: **Yes / No**

4. Mini Project evaluation matrix

(Tick the appropriate box: 1 is lowest and 5 is the highest)

Dimension	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
-----------	--------	--------	--------	--------	--------

---	---	---	---	---	---
-----	-----	-----	-----	-----	-----

**Communication
ability**

**Organization of
material**

**Response to
review questions**

Report structure

and format**

**Technical content
of the report**

**Explanation on
the significance of
the assignment**

**Analysis of
alternative
approaches**

**Recommendation
and suggestions**

**Discussion on
benefits and
limitations**

**Understanding of
the subject of Mini
Project**

**Creative thinking
ability to come up
with new ideas**

**Report
submitted**

**Viva / Seminar
presentation**

- **Student abilities in general:** _____
 - **Any other comments:** _____
- **Date:** _____
- **Signature of Supervisor:** _____